Esercizi Laboratorio Sistemi operativi (**clicca per indice**)

Laboratorio 01

Esercizio 1: Processi vs Thread

Obiettivo: Sfruttare una differenza tra processi e thread nella **gestione della memoria** per rendere ancora meno reattiva la versione multi-processo, a vantaggio di quella multi-thread, e verificare l'aumento dello speedup.

Come funzionano i programmi di misura:
- Sorgenti: **reactivity_processes.c** e **reactivity-threads.c**.
- Per N volte si crea un processo (**fork** + **wait**) o un thread (**pthread_create** + **pthread_join**) che esegue **do_work()** e terminated.
- Si misura il tempo medio di lancio + terminazione con la libreria **performance**.
- Lo speedup è calcolato come:

$$\text{speedup} = \frac{\text{tempo reactivity process}}{\text{tempo thread process}}$$

- Nei sorgenti originali **do_work()** è vuota, quindi la differenza tra processi e thread è piccola.

Differenza da sfruttare:
- Con **fork()** il processo figlio riceve una copia dell'intera memoria del padre. Su Linux la copia avviene in **copy-on-write**: una pagina viene duplicata solo quando il figlio la scrive;
- I thread di un processo condividono la memoria, quindi non hanno questo costo.

Attenzione (**copy-on-write**):
- Con **fork()** il figlio lavora su una copia dello spazio di indirizzamento del padre.
- I thread di uno stesso processo condividono lo spazio di indirizzamento.

Idea:
Far scrivere a **do_work()** su un grande buffer globale, una volta per ogni pagina:
- **processi**: ogni figlio paga il costo di ogni pagina che tocca (page fault + pagina nuova o copiata);
- **thread**: le pagine sono condivise, il costo si paga una volta sola.

do_work() va modificata allo stesso modo nei due sorgenti.

Soluzione variante 1: buffer allocato nel main con calloc

Codice: Definizioni e **do_work()**, identiche in reactivity-processes.c e reactivity-thread.c:

```
#define ITEMS (1 << 24)   // 2^24 int = 64 MB
#define STEP  1024        // 1024 int = 4 KB
int* global_buff = NULL;

void do_work() {
    int j;
    for (j = 0; j < ITEMS; j += STEP) {
        global_buff[j] = j;   // una scrittura per ogni pagina da 4 KB
    }
}
```

Nel main, prima delle misure:

```
global_buff = (int*)calloc(ITEMS, sizeof(int));
if (global_buff == NULL) { ... }
... // misure invariate
```

Spiegazione del codice:
- **ITEMS** è la dimensione del buffer (64 MB di **int**), abbastanza grande da rendere visibile il costo del **fork()**.
- **STEP = 1024** int = 4 KB = una pagina: il ciclo avanza di una pagina alla volta e scrive un valore per pagina, quindi tocca tutte le 16.384 pagine con il minio lavoro. Il lavoro in sé è trascurabile: conta l'effetto sulla memoria.
- Il buffer è allocato e azzerato nel **main** prima delle misure, quindi la **calloc** non entra nei tempi.

Cosa succede nei due casi:
- **Processi**: il padre non scrive mai nelle pagelle, quindi il kernel le mappa come pagine azzerate, allocate solo alla prima scrittura. Ogni figlio, a ogni **fork()**, alloca 16.384 pagine nuove (page fault + pagina nuova).
- **Thread**: le pagine sono allocate dal primo thread e poi condivise da tutti. Non c'è page fault né copia a ogni lancio; il ciclo di scritture costa comunque qualcosa, ma molto meno.

Alternativa alla variante 1: malloc + memset

Cambia un solo dettaglio rispetto alla variante 1: come viene preparato il buffer. Invece della **calloc**, il padre alloca con **malloc** e azzera con **memset,** sempre nel **main** e prima delle misure (**do_work()** resta identica).

```
#include <string.h>

global_buff = (int*)malloc(ITEMS * sizeof(int));
if (global_buff == NULL) { ... }
memset(global_buff, 0, ITEMS * sizeof(int));   // il padre scrive in tutte le pagine
... // misure invariate
```

Così il padre tocca tutte le pagine, che quindi esistono già in memoria fisica al momento del **fork()**:
- **Processi**: i figli le copiano alla scrittura (vero copy-on-write: page fault + copia della pagina). In più **fork()** deve copiare le tabelle delle pagine del padre, e il costo cresce con la memoria mappata;
- **Thread**: nessun costo di allocazione, le pagine sono già condivise.

Soluzione, variante 2: modificare solo do_work()

Vincolo: non toccare il **main** (sono ammesse variabili globali). Si usa un array globale non inizializzato:

```
#define ITEMS (1 << 24)   // 2^24 int = 64 MB
#define STEP  1024        // 1024 int = 4 KB
int global_buff[ITEMS];   // non inizializzato: niente calloc, nessuna modifica al main

void do_work() {
    int j;
    for (j = 0; j < ITEMS; j += STEP)
        global_buff[j] = j;   // una scrittura per pagina
}
```

- Il kernel mappa **global_buff[ITEMS]** come pagine azzerate, allocate solo alla prima scrittura: stesso comportamento della **calloc** da 64 MB.
- **Processi**: il padre non lo tocca mai, quindi ogni figlio alloca 16.384 pagine nuove a ogni **fork()**.
- **Thread**: pagine allocate dal primo thread, poi condivise da tutti.
- Il **main** è identico a quello del codice di partenza.

Quanto conta il valore di STEP (N=300):
- **STEP = 1**: anche i thread fanno 16M scritture, il cui costo domina in entrambi i casi. quindi lo speedup è minimo.
- **STEP < 1024**: le pagine da duplicare sono le stesse ma il numero di scritture per pagina si riduce, quindi il costo delle copie conta sempre di più.
- **STEP = 1024** (4 KB / **sizeof(int)**): una sola scrittura per pagina, quindi il rapporto tra costo delle copie e costo delle scritture è massimo.
- **STEP > 1024**: si saltano pagine, quindi meno copie, e i costi fissi di **fork()**/**exit** pesano di più, quindi lo speedup cala.

Compilazione ed esecuzione:
Con N basso, perché ogni **fork()** è ora lento:

```
gcc -o reactivity-processes reactivity-processes.c performance.c -lpthread -lrt -lm
gcc -o reactivity-threads reactivity-threads.c performance.c -lpthread -lrt -lm
./reactivity-processes <N>
./reactivity-threads <N>
```

Per la variante 2 la compilazione e l'esecuzione sono invariate, con i sorgenti modificati.

Risultati: Misurati con l'implementazione **malloc** + **memset** (256 MB, un byte per pagina), sulla macchina di prova con 2 core. Sulla tua macchina i valori assoluti saranno diversi.

	Processi	Thread	Speedup
Versione originale (N = 1000)	197 µs	67 µs	≈ 2,9
Versione modificata (N = 200)	258.619 µs	2.933 µs	≈ 88

Conclusione: Facendo lavorare i processi figli su una grande quantità di memoria, il costo del copy-on-write (o dell'allocazione della pagine) fa crescere molto il tempo dei processi. I thread, che condividono lo spazio di indirizzamento, lo pagano massimo si ottiene con una scrittura per pagine (**STEP = 1024**).

Esercizio 2: Accesso concorrente a variabili condivise

Obiettivo: Capire cosa succede quando più thread scrivono in concorrenza su una variabile condivisa, e risolvere il problema senza meccanismi di sincronizzazione, lavorando sulle strutture dati.
- Sorgente di partenze: **concurrent_threads.c**.
- Il programma lancia **N thread** in parallelo; ciascuno aggiunge **M volte un valore V** a una variabile condivisa, inizializzata a 0.
- Al termine il **main** verifica che gli incrementi equivalgono complessivamente a $N \cdot M \cdot V$.

Il problema (**add** perse): Se tutti i thread incrementano la stessa variabile, l'operazione $+=$ non è atomica: ogni thread legge il valore, lo somma e lo riscrive, e se due thread leggono lo stesso valore prima che uno dei due riscriva, un incremento va perso. E' una **race condition**.

Esempio. (N = 1000, M = 10000, V = 1)

```
$ ./concurrent_threads 1000 10000 1
The value of the shared variable is 8674815. It should have been 10000000
Number of lost adds: 1325185
```

Il valore atteso è 10.000.000, ma ne risultano circa 1,3 milioni in meno.

Soluzione 1: una cella per thread

L'idea è far si che **nessun thread scriva sulla stessa locazione di un altro**:
- ogni thread incrementa una locazione di memoria diversa (una cella di un array);
- alla fine il **main** somma tutti i valori.

Sorgente: **sol-concurrent_threads.c**

```
unsigned long int* shared_array;    // una cella per thread

void* thread_work(void *arg) {
    int thread_idx = *((int*)arg);  // legge il proprio indice dall'argomento
    int i;
    for (i = 0; i < m; i++)
        shared_array[thread_idx] += v;  // scrive SOLO nella propria cella
    return NULL;
}

// main
shared_array = calloc(n, sizeof(unsigned long int));   // n celle azzerate
int* thread_ids = (int*)malloc(n * sizeof(int));       // un indice per thread
for (i = 0; i < n; i++) {
    thread_ids[i] = i;
    pthread_create(&threads[i], NULL, thread_work, &thread_ids[i]);  // indirizzo diverso per ogni thread
}
for (i = 0; i < n; i++) {
    pthread_join(threads[i], NULL);          // aspetta il thread i
    computed_value += shared_array[i];       // legge la cella i solo DOPO la join
}
```

Spiegazione passo per passo:
1. **calloc** crea un array con **una cella per thread**, già azzerata.
2. Ogni thread riceve un **indirizzo diverso** (**&thread_ids[i]**), da cui legge il proprio indice, e scrive **solo nella propria cella**: nessuna scrittura concorrente.
3. Il **main** legge la cella **i** solo **dopo la pthread_join** del thread **i**, quindi il thread ha già finito di scrivere.
4. Il **main** somma tutte le celle in **computed_value**.

Compilazione ed esecuzione:

```
gcc -o sol-concurrent_threads sol-concurrent_threads.c -lpthread
./sol-concurrent_threads <N> <M> <V>
```

Non dovrebbero risultare **add** perse.

Esempio. (N = 1000, M = 10000, V = 1)

```
$ ./sol-concurrent_threads 1000 10000 1
The value computed on the array is 10000000. It should have been 10000000
```

Cosa succede se al posto di &thread_ids[i] si usa &i?
Il codice sembra funzionare, ma è sbagliato:
- non c'è alcuna garanzia su **quando** verrà eseguita l'istruzione **int thread_idx = *((int*)arg);**;
- nel frattempo il valore nella locazione puntata da **arg** può cambiare: è il contatore **i**, che il **main** continua a incrementare (e riusa nel ciclo di join);
- il risultato è che più thread possono avere la stessa identità;
- si propone il problema dell'accesso concorrente.

Soluzione alternativa: valore restituito con pthread_exit

- Ogni thread accumula su una **variabile propria** e restituisce il valore tramite **pthread_exit**.
- Il **main** raccoglie i valori tramite **pthread_join** e li somma,
- Non servono già ne l'array condiviso ne gli indici dei thread.

Sorgente: **sol2-concurrent_threads.c**

```
void* thread_work(void *arg) {
    unsigned long* portion = calloc(1, sizeof(unsigned long));  // risultato nell'heap
    int i;
    for (i = 0; i < m; i++)
        *portion += v;              // accumula sulla variabile propria
    pthread_exit(portion);          // restituisce il puntatore al risultato
}

// main
for (i = 0; i < n; i++)
    pthread_create(&threads[i], NULL, thread_work, NULL);   // nessun argomento

unsigned long computed_value = 0;
unsigned long* thread_result;
for (i = 0; i < n; i++) {
    pthread_join(threads[i], (void**) &thread_result);  // riceve il puntatore passato a
pthread_exit
    computed_value += *thread_result;
    free(thread_result);                        // il main libera la memoria
}
```

Note:
- Il **risultato va nell'heap**: una variabile locale vive nello stack del thread, che non esiste più dopo la sua terminazione.
- **Nessun argomento**: ogni thread non ha bisogno di sapere chi è.
- **pthread_join** restituisce il puntatore passato a **pthread_exit**; il **main** lo libera con **free**.

Compilazione ed esecuzione:

```
gcc -o sol2-concurrent_threads sol2-concurrent_threads.c -lpthread
./sol2-concurrent_threads <N> <M> <V>
```

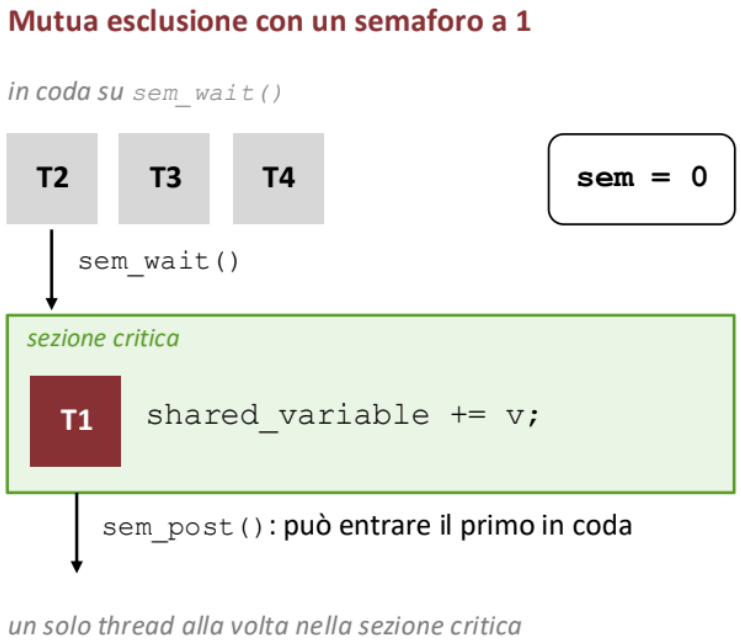
Conclusione: Per evitare le add perse senza meccanismi di sincronizzazione basta che **nessuna locazione sia scritta da più di un thread**: o dando a ogni thread una cella dell'array condiviso (soluzione 1), oppure facendo accumulare ogni thread su una variabile propria e raccogliendo i

risultato con **pthread_join** (soluzione alternativa). In entrambi i casi il **main** somma i valori solo dopo la **join**, e il risultato è esattamente $N \cdot M \cdot V$.

Laboratorio 02

Esercizio 1: Accesso alla sezione critica in mutua esclusione

Il problema di partenza:
Riprendiamo **concurrent_threads.c** (Lab01-es2):
- N thread aggiungono **M** volte il valore **V** a una variabile condivisa **shared_variable**.
- Il risultato attesa è quindi $N \cdot M \cdot V$ (con N = 1000, M = 10000, V = 1 sono 10.000.000).
- L'istruzione **shared_variable += v;** è la sezione critica. Non è atomica, perché si scompone in tre passi: lettura, somma, scrittura.
- Se due thread leggono lo stesso valore prima che uno dei due scriva, uno dei due incrementi va perso. Sono le add perse, una race condition (nelle slide: 457.001 add perse su 10.000.000).



L'idea della soluzione:
Si usa un semaforo inizializzato a 1, che si comporta come un lock.
- **sem_wait()** prima della sezione critica:
  - se il valore è 1, lo porta a 0 e il thread entra;
  - se il valore è 0, il thread si blocca in attesa.
- **sem_post()** all'uscita:
  - rimette il valore a 1, oppure sveglia un thread in attesa.
- Così un solo thread alla volta esegue **shared_variable += v;** e non ci sono add perse.

Passo per passo:
1. Copiare **concurrent_threads.c** in **concurrent_threads_semaphore.c**.
2. Includere **<semaphore.h>**.
3. Dichiarare il semaforo come variabile globale (**sem_t sem;**), perché tutti i thread devono usare lo stesso semaforo.
4. Nel **main**, inizializzarlo con **sem_init(&sem, 0, 1)**:
   - **0** (**pshared**): il semaforo è condiviso fra i thread dello stesso processo;
   - **1**: valore iniziale, che da la mutua esclusione.

5. In **thread_work**, racchiude solo l'incremento tra **sem_wait** e **sem_post**, a ogni iterazione.
6. Misurare il tempo con la libreria **performance** (**begin** / **end** attorno a creazione e join dei thread).
7. A fine programma, distruggere il semaforo con **sem_destroy**.

Codice:

```
#include "performance.h"   // libreria per misurare il tempo (timer, begin, end, get_milliseconds)
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
#include <semaphore.h>     // sem_t, sem_init, sem_wait, sem_post, sem_destroy
#include <errno.h>         // errno, per stampare il codice d'errore

#define N 1000   // numero di thread
#define M 10000  // iterazioni per thread
#define V 1      // valore sommato a ogni iterazione

sem_t sem;                      // GLOBALE: tutti i thread usano lo stesso semaforo
unsigned long int shared_variable; // la variabile condivisa (risorsa critica)
int n = N, m = M, v = V;

void* thread_work(void *arg) {
    int i;
    for (i = 0; i < m; i++) {
        // ACQUISIZIONE del lock: se sem vale 1 lo porta a 0 e prosegue,
        // se vale 0 il thread si blocca finché qualcuno fa sem_post
        if (sem_wait(&sem) != 0) {
            fprintf(stderr, "Can't lock the semaphore, error %d\n", errno);
            return NULL;
        }

        // SEZIONE CRITICA: un solo thread alla volta arriva qui
        shared_variable += v;

        // RILASCIO del lock: rimette sem a 1 oppure sveglia un thread in attesa
        if (sem_post(&sem) != 0) {
            fprintf(stderr, "Can't unlock the semaphore, error %d\n", errno);
            return NULL;
        }
    }
    return NULL;
}

int main(int argc, char **argv)
{
    // parametri opzionali da riga di comando: N, M, V
    if (argc > 1) n = atoi(argv[1]);
    if (argc > 2) m = atoi(argv[2]);
    if (argc > 3) v = atoi(argv[3]);
    shared_variable = 0;
    timer t;                    // oggetto per la misura del tempo

    printf("Going to start %d threads, each adding %d times %d to a shared variable initialized
           to zero...", n, m, v); fflush(stdout);
    pthread_t* threads = (pthread_t*)malloc(n * sizeof(pthread_t));
    int i;
```

```
    // pshared = 0 -> condiviso fra i thread del processo
    // valore iniziale 1 -> si comporta come un lock (mutua esclusione)
    if (sem_init(&sem, 0, 1) != 0) {
        fprintf(stderr, "Can't initialize the semaphore, error %d\n", errno);
        exit(EXIT_FAILURE);
    }

    begin(&t);                  // INIZIO misura del tempo
    for (i = 0; i < n; i++)       // creazione dei thread (invariata)
        if (pthread_create(&threads[i], NULL, thread_work, NULL) != 0) {
            fprintf(stderr, "Can't create a new thread, error %d\n", errno);
            exit(EXIT_FAILURE);
        }
    printf("ok\n");

    printf("Waiting for the termination of all the %d threads...", n); fflush(stdout);
    for (i = 0; i < n; i++)       // join dei thread (invariato)
        pthread_join(threads[i], NULL);
    end(&t);                    // FINE misura del tempo
    printf("ok\n");

    // verifica del risultato: atteso = N * M * V
    unsigned long int expected_value = (unsigned long int)n*m*v;
    printf("The value of the shared variable is %lu. It should have been %lu\n", shared_variable,
           expected_value);
    if (expected_value > shared_variable) {
        unsigned long int lost_adds = (expected_value - shared_variable) / v;
        printf("Number of lost adds: %lu\n", lost_adds);
    }
    printf("It took %lu milliseconds\n", get_milliseconds(&t));

    free(threads);
    sem_destroy(&sem);          // si distrugge il semaforo quando non serve più
    return EXIT_SUCCESS;
}
```

Compilazione ed esecuzione:

```
gcc -o concurrent_threads_semaphore concurrent_threads_semaphore.c \
    performance.c -lpthread -lrt -lm

./concurrent_threads_semaphore <N> <M> <V>
```

Confronto delle prestazioni:
Esempio con N = 1000, M = 10000, V = 1:

Soluzione	Valore finale	Tempo
Lab01 (una cella per thread, senza semafori)	10000000 (corretto)	43 ms
Con semaforo	10000000 (corretto)	1605 ms

Il semaforo elimina le add perse, ma il tempo di esecuzione aumenta rispetto all soluzione senza semafori.

Questo succede perché si fanno $N \cdot M = 10^7$ coppie di **sem_wait** / **sem_post**, una o ogni iterazione.

Riepilogo:
- Problema: **shared_variable += v;** non è atomica, quindi con più thread si hanno race condition e add perse.
- Sezione critica: solo l'incremento **shared_variable += v;**.
- Soluzione: un semaforo globale inizializzato a 1 (**sem_init(&sem, 0, 1)**), che funziona da lock.
- Protocollo: **sem_wait** (acquisisco) → sezione critica → **sem_post** (rilascio), a ogni iterazione.
- Risultato: nessuna add persa, valore finale uguale a $N \cdot M \cdot V$.
- Costo: tempo di esecuzione molto più alto (1605 ms contro 43 ms nell'esempio).
- Ricordarsi di: **sem_destroy** a fine programma e di compilare con **-lpthread -lrt** -**lm**.

Esercizio 2: Accesso in mutua esclusione a N risorse

Il problema:
- Ci sono N risorse, ognuna usabile in mutua esclusione (es. un pool di connessioni a DB, un pool di thread).
- M thread in concorrenza devono accedervi, con M > N.
- Come si fa con i semafori? Suggerimento: prima solo un thread alla volta poteva entrare nella sezione critica, ora vogliamo che possano entrare N thread alla volta.

Soluzione base:
Si inizializza il semaforo a N invece che a 1. Il codice è **scheduler.c**, la soluzione è **scheduler_solution.c**. Come si usa:
- Si lancia **./scheduler**.
- Premendo INVIO vengono lanciati **THREAD_BURST** thread, che tentano di accedere in parallelo a **NUM_RESOURCES** risorse e le usano per processare ciascuno **NUM_TASKS** work item.
- Un work item richiede un tempo random tra 0 e **MAX_SLEEP** secondi.
- Premendo **CTRL+D** il programma termina.
- Si osserva l'interleaving dei thread e il fatto che non ci sono mai più di **NUM_RESOURCES** thread con accesso a una risorsa nello stesso memento.

Perché funziona:
- I primi N thread che eseguono **sem_wait()** vedono un valore non negativo dopo il decremento e accedono alla sezione critica.
- I thread successivi decremento (valore negativo) e vengono messi in coda.
- Quando uno dei primi N thread fa **sem_post()**, il semaforo viene incrementato. Siccome il valore era negativo, al massimo diventa 0, quindi un thread in coda viene svegliato e gli altri restano in attesa.
- Ad ogni successiva **sem_post()** il semaforo viene incrementato. Finché non diventa positivo, ci sono thread in coda che vengono svegliati a ogni **sem_post()**.

Esecuzione reale: `NUM_RESOURCES` = 3, un burst di 5 thread



Finché c'è qualcuno in coda, ogni **sem_post()** sveglia un thread che prende la risorsa: il valore torna subito a 0 e risale solo quando la coda è vuota.

Codice (**scheduler_solution.c**):

```
#include <errno.h>      // errno, per capire il tipo di errore
#include <pthread.h>
#include <semaphore.h>  // sem_t, sem_init, sem_wait, sem_post, sem_destroy
#include <stdio.h>
#include <stdlib.h>
#include <string.h>     // strerror(): trasforma errno in una stringa leggibile
#include <unistd.h>     // sleep()

#define MAX_SLEEP       3   // durata massima di un work item (simulato con sleep)
#define NUM_RESOURCES   3   // numero di risorse speciali disponibili
#define NUM_TASKS       3   // work item per thread
#define THREAD_BURST    5   // quanti thread vengono lanciati a ogni INVIO

// Struttura che racchiude gli argomenti di ogni thread
typedef struct thread_args_s {
    int     ID;
    sem_t*  semaphore;      // tutti i thread ricevono lo STESSO puntatore al semaforo
    int     num_tasks;
} thread_args_t;

void* client(void* arg_ptr) {
    thread_args_t* args = (thread_args_t*) arg_ptr;
    int i, ret = 0;

    /*** ACQUISIZIONE della risorsa ***/
    // se ci sono già NUM_RESOURCES thread con una risorsa, questo thread resta in coda
    ret = sem_wait(args->semaphore);
    if (ret) {
        fprintf(stderr,"[FATAL ERROR] Could not lock the semaphore from thread %d: %s\n", args-
>ID, strerror(errno));
        exit(1);
    }

    printf("[@Thread%d] Resource acquired...\n", args->ID);

    /*** Uso della risorsa: tutti i work item, ininterrottamente ***/
    for (i = 0; i < args->num_tasks; ++i) {
        // simuliamo un work item dormendo da 0 a MAX_SLEEP secondi
        sleep(rand() % (MAX_SLEEP+1));
    }

    /*** RILASCIO della risorsa ***/
    // il valore del semaforo risale, oppure parte uno dei thread in coda
    ret = sem_post(args->semaphore);
    if (ret) {
        fprintf(stderr,"[FATAL ERROR] Could not unlock the semaphore from thread %d: %s\n",
args->ID, strerror(errno));
        exit(1);
    }
            if (pthread_create(&thread_handle, NULL, client, args)) {
    printf("[@Thread%d] Done. Resource released!\n", args->ID);

    free(args);       // ogni thread libera i propri argomenti
```

```
    return NULL;
}

int main(int argc, char* argv[]) {
    printf("Welcome! This is a very simple resource scheduler.\n\n");
    printf("We are simulating a system with %d available special resources. Hence, no more "
           "than %d threads can get exclusive access to them at the same time.\n\n",
NUM_RESOURCES, NUM_RESOURCES);

    int ret = 0;
    int thread_ID = 0;

    sem_t* semaphore = malloc(sizeof(sem_t));   // semaforo allocato sull'heap

    /*** Inizializzazione: valore iniziale NUM_RESOURCES (=3) invece di 1 ***/
    // pshared = 0 -> semaforo condiviso fra i thread del processo
    // fino a 3 thread contemporaneamente in "sezione critica"
    ret = sem_init(semaphore, 0, NUM_RESOURCES);
    if (ret) {
        fprintf(stderr,"[FATAL ERROR] Could not create a semaphore: %s\n", strerror(errno));
        exit(1);
    }

    /* Ciclo principale */
    printf("[DRIVER] Press ENTER to spawn %d new threads. Press CTRL+D to quit!\n",
THREAD_BURST);

    while(1) {
        int input_char;

        // si scartano tutti i caratteri tranne '\n' (INVIO) ed EOF (CTRL+D)
        while ( (input_char = getchar()) != '\n' && input_char != EOF ) continue;

        if (input_char == EOF) break;   // CTRL+D: si esce dal ciclo

        printf("==> [DRIVER] Spawning %d threads now...\n", THREAD_BURST);

        int i;
        for (i = 0; i < THREAD_BURST; ++i) {     // burst di thread
            pthread_t thread_handle;

            thread_args_t* args = malloc(sizeof(thread_args_t));
            args->semaphore = semaphore;          // stesso semaforo per tutti
            args->ID = thread_ID;
            args->num_tasks = NUM_TASKS;

            if (pthread_create(&thread_handle, NULL, client, args)) {
                fprintf(stderr,"==> [DRIVER] FATAL ERROR: cannot create thread %d: %s\nExiting...
\n", thread_ID, strerror(errno));
                exit(1);
            }

            ++thread_ID;

            // non aspettiamo questo thread: lo stacchiamo (detach)
            pthread_detach(thread_handle);
        }
        printf("==> [DRIVER] Press ENTER to spawn %d new threads. Press CTRL+D to quit!\n",
THREAD_BURST);
```

```
    }
    printf("Exiting...\n");

    /*** Si distrugge il semaforo e si libera la memoria ***/
    sem_destroy(semaphore);
    free(semaphore);

    pthread_exit(NULL);
}
```

Compilazione ed esecuzione:

```
gcc -o scheduler_solution scheduler_solution.c -lpthread
./scheduler_solution
```

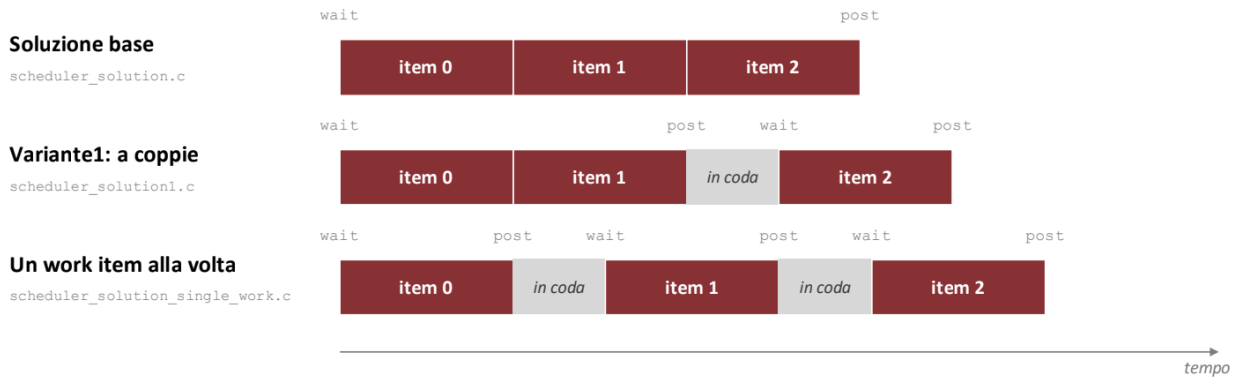
Output reale (**MAX_SLEEP = 3**, **NUM_TASKS = 3**):

```
 0 s  ==> [DRIVER] Spawning 5 threads now...
 0 s  [@Thread0] Resource acquired...
 0 s  [@Thread2] Resource acquired...
 0 s  [@Thread1] Resource acquired...
 4 s  [@Thread1] Done. Resource released!
 4 s  [@Thread3] Resource acquired...
 5 s  [@Thread2] Done. Resource released!
 5 s  [@Thread4] Resource acquired...
 9 s  [@Thread0] Done. Resource released!
10 s  [@Thread3] Done. Resource released!
11 s  [@Thread4] Done. Resource released!
```

- Le prime 3 richieste trovano una risorsa libera: il semaforo passa da 3 a 0.
- Thread3 e Thread4 erano già in coda e entrano solo quando qualcuno rilascia (t=4s e t=5s).
- Mai più di 3 thread tra “acquired” e “released”

Variante 1: rilascio ogni coppia di work item
Richiesta: invece di usare la risorsa per tutti i work item di fila, ogni thread deve rilasciarla dopo aver completato una coppia di work item e rimettersi in coda per riottenerla. Una volta riacquista, completa la coppia successiva, e così via. Alla fine rilascia definitivamente la risorsa dopo che tutti i work item sono stati processati.

Con **NUM_TASK = 3**, quando il thread possiede la risorsa:



Nella variante a coppie la risorsa viene rilasciata ogni 2 work item e il thread si rimette in coda, quindi gli altri thread entrano prima. Con **NUM_TASK = 3** l'ultima "coppia" ha un solo work item: la risorsa va rilasciata comunque.

Codice (**client** di **scheduler_solution1.c**):

```
void* client(void* arg_ptr) {
    thread_args_t* args = (thread_args_t*) arg_ptr;
    int i, ret = 0;

    for (i = 0; i < args->num_tasks; ++i) {

        // i PARI (0, 2, ...) sono l'inizio di una coppia: si ACQUISISCE la risorsa
        if (i%2 == 0){
            ret = sem_wait(args->semaphore);
            if (ret) {
                printf("[FATAL ERROR] Could not lock the semaphore from thread %d: %s\n",
                        args->ID, strerror(errno));
                exit(1);
            }
            printf("[@Thread%d] Resource acquired...\n", args->ID);
        }

        // work item simulato
        printf("[@Thread%d] Task %d...\n", args->ID, i);
        sleep(rand() % (MAX_SLEEP+1));

        // i DISPARI (fine coppia) OPPURE l'ULTIMO work item: si RILASCIA la risorsa
        if (i%2 == 1 || i == args->num_tasks-1){
            ret = sem_post(args->semaphore);
            if (ret) {
                printf("[FATAL ERROR] Could not unlock the semaphore from thread %d: %s\n",
                        args->ID, strerror(errno));
                exit(1);
            }
            printf("[@Thread%d] Done. Resource released!\n", args->ID);
        }
    }

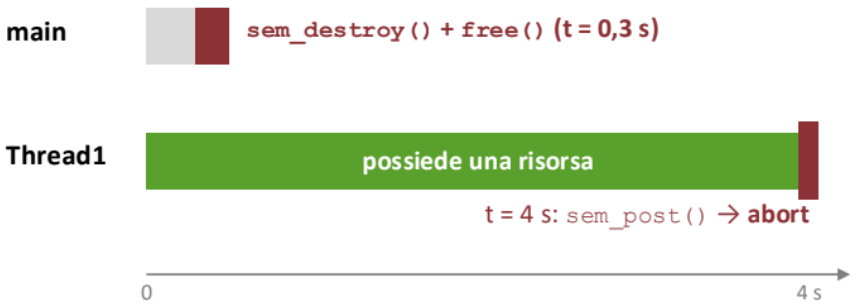
    free(args);
    return NULL;
}
```

ATTENZIONE: senza **i == args->num_task-1**, con **NUM_TASKS** dispari ogni thread terminerebbe tenendo una risorsa. Dopo 3 thread le risorse sono finite e gli altri restano bloccati per sempre.

Cosa si osserva nell'esecuzione:
- Thread1 rilascia e si riprende subito la risorsa, anche se Thread3 e Thread4 sono in coda da t=0 (idem Thread2).
- **sem_post()** sveglia un thread in coda ma non gli "consegna" la risorsa: chi chiama subito **sem_wait()** può arrivare prima.
- I semafori POSIX non garantiscono l'ordine FIFO: in casi sfavorevoli un thread può restare in coda a lungo (starvation).

Variante 2: chiusura del main prima dei thread
Il problema: Se si aumenta sensibilmente il tempo della **sleep** e si preme CTRL+D subito dopo un burst:
- il main esce dal ciclo e chiama **sem_destroy()** e **free()** sul semaforo;

- i thread però sono ancora vivi: alcuni lavorano, altri sono in coda su **sem_wait()**;
- la loro **sem_post()** agisce su un semaforo distrutto, in memoria già liberata → comportamento indefinito.



Idea: una variabile che conta quanti thread sono attualmente aperti.
- Il main la incrementa quando crea un thread.
- Il thread la decrementa prima di terminare.
- Solo quando vale 0 si può chiudere e distruggere il semaforo.

Problemi:
- l'accesso alla variabile può causare problemi di concorrenza, quindi serve un semaforo per la mutua esclusione;
- la soluzione comporta busy waiting del main.

Esistono due modi per risolvere senza problemi di mutua esclusione.

Soluzione 1: un secondo semaforo come contatore (**scheduler_solution2.c**)

```
sem_t semaphore2;   // semaforo usato come CONTATORE dei thread terminati
                    // (nessuna variabile da proteggere)

void* client(void* arg_ptr) {
    thread_args_t* args = (thread_args_t*) arg_ptr;
    int i, ret = 0;

    ret = sem_wait(args->semaphore);     // acquisizione risorsa (come nella soluzione base)
    /* ...controllo errori... */
    printf("[@Thread%d] Resource acquired...\n", args->ID);

    for (i = 0; i < args->num_tasks; ++i) {
        sleep(rand() % (MAX_SLEEP+1));   // work item
    }

    ret = sem_post(args->semaphore);     // rilascio risorsa
    /* ...controllo errori... */
    printf("[@Thread%d] Done. Resource released!\n", args->ID);

    free(args);

    // ULTIMA operazione del thread: deposita un "gettone" su semaphore2
    // per comunicare al main che ha finito
    ret = sem_post(&semaphore2);
    /* ...controllo errori... */

    return NULL;
}
```

Nel main:

```
// valore iniziale 0: nessun thread ancora terminato
ret = sem_init(&semaphore2, 0, 0);

/* ... ciclo con i burst invariato (++thread_ID per ogni thread creato) ... */

printf("[Driver] Waiting thread termination...\n");

// il main raccoglie UN gettone per OGNI thread creato:
// si blocca su sem_wait senza consumare CPU (0% con top)
for (j = 0; j < thread_ID; ++j) {
    ret = sem_wait(&semaphore2);
    /* ...controllo errori... */
}

printf("[Driver] Exiting...\n");

// ora tutti i thread sono terminati: si può distruggere in sicurezza
sem_destroy(semaphore);
sem_destroy(&semaphore2);
free(semaphore);
```

Soluzione 2: on_exit() (**scheduler_solution2b.c**)

```
// funzione eseguita alla TERMINAZIONE DEL PROCESSO
// (il semaforo viene distrutto quando nessuno lo usa più: non serve contare i thread)
void exit_safely(int status, void *arg){
    printf("Exit safely...\n");
    sem_t* semaphore = (sem_t*) arg;

    fflush(NULL);              // svuota i buffer di output

    sem_destroy(semaphore);
    free(semaphore);
}

int main(int argc, char* argv[]) {
    /* ... inizializzazione e burst invariati ... */

    printf("Exiting...\n");

    // on_exit() REGISTRA la funzione da eseguire alla terminazione del processo
    on_exit(exit_safely, semaphore);

    // pthread_exit() nel main termina SOLO il thread main:
    // il processo vive finché c'è almeno un thread
    pthread_exit(NULL);
}
```

Esecuzione:

```
 0,3 s  Exiting...
 4 s    [@Thread2] Done. Resource released!
 (...)
 11 s   [@Thread3] Done. Resource released!
 11 s   Exit safely...        ← eseguita dopo l'ultimo thread (t = 11 s)
```

on_exit() è un'estensione GNU. In C standard c'è **atexit()**, senza argomento (quindi serve un semaforo globale).

Variante alternativa: 2 risorse contemporaneamente
Richiesta: ogni thread ha bisogno di 2 risorse contemporaneamente per processare i suoi work item (es. una connessione in lettura e una in scrittura, un buffer di input e uno di output). Si parte da **scheduler_solution.c**, con una nuova costante **RES_PER_THREAD = 2** (quindi al più 2 thread alla volta al lavoro). Le risorse si rilasciano tutte insieme dopo l'ultimo work item. +

Primo tentativo (**SBAGLIATO**): **risorse prese una alla volta**

```
// SBAGLIATO: le risorse prese una alla volta
for (i = 0; i < RES_PER_THREAD; ++i) {
    sem_wait(args->semaphore);
    printf("... Resource %d/%d acquired...\n", ...);
    if (i < RES_PER_THREAD - 1)
        usleep(ACQUIRE_DELAY_US);   // pausa: 0,1 s (rende visibile l'interleaving)
}
... // work item
for (i = 0; i < RES_PER_THREAD; ++i)
    sem_post(args->semaphore);
```

Con 4 risorse e 5 thread: se i primi 4 prendono una risorsa a testa, nessuno arriva alla seconda.
- T0 - T3 aspettano la seconda risorsa, T4 aspetta la prima.
- Nessuno rilascerà mai la propria risorsa → deadlock.
- Ci sono tutte le condizioni del deadlock: mutua esclusione, hold and wait, nessuna prelazione, attesa circolare.

Soluzione: un mutex per l'acquisizione (**scheduler_solution3.c**)

```
sem_t acquire_mutex;   // semaforo a 1 (mutex): rende ATOMICA l'acquisizione delle 2 risorse

void* client(void* arg_ptr) {
    thread_args_t* args = (thread_args_t*) arg_ptr;
    int i, ret = 0;

    /*** Fase di acquisizione: un solo thread alla volta prende le sue 2 risorse ***/
    ret = sem_wait(&acquire_mutex);
    /* ...controllo errori... */

    for (i = 0; i < RES_PER_THREAD; ++i) {
        ret = sem_wait(args->semaphore);   // una risorsa alla volta, ma DENTRO il mutex
        /* ...controllo errori... */
        printf("[@Thread%d] Resource %d/%d acquired...\n", args->ID, i + 1, RES_PER_THREAD);
        if (i < RES_PER_THREAD - 1)
            usleep(ACQUIRE_DELAY_US);      // pausa per rendere visibile l'interleaving
    }

    // il mutex si rilascia APPENA ottenute ENTRAMBE le risorse, non alla fine del lavoro
    ret = sem_post(&acquire_mutex);
    /* ...controllo errori... */

    /*** Work item ***/
    for (i = 0; i < args->num_tasks; ++i) {
        sleep(rand() % (MAX_SLEEP+1));
    }
```

```
    /*** Rilascio: NON usa il mutex, chi lavora può sempre restituire le risorse ***/
    for (i = 0; i < RES_PER_THREAD; ++i) {
        ret = sem_post(args->semaphore);
        /* ...controllo errori... */
    }

    printf("[@Thread%d] Done. Resources released!\n", args->ID);

    free(args);
    return NULL;
}
```

Nel main, oltre al semaforo a **NUM_RESOURCES** (qui 4):

```
ret = sem_init(&acquire_mutex, 0, 1);   // valore 1 -> mutua esclusione
...
sem_destroy(&acquire_mutex);
```

Non c'è più deadlock.

Esecuzione reale (4 risorse, 2 a testa):

```
0,0 s  [@Thread0] Resource 1/2 acquired...
0,1 s  [@Thread0] Resource 2/2 acquired...
0,1 s  [@Thread1] Resource 1/2 acquired...     ← Thread1 inizia solo dopo che Thread0 ha preso
ENTRAMBE
0,2 s  [@Thread1] Resource 2/2 acquired...
4,2 s  [@Thread1] Done. Resources released!
4,2 s  [@Thread2] Resource 1/2 acquired...
...
```

- Thread1 inizia ad acquisire solo quando Thread0 ha preso entrambe le risorse.
- Al più un thread alla volta tiene una risorsa e ne aspetta un'altra: quello che possiede il mutex.
- Aspetta solo risorsa di thread che hanno già tutto e prima o poi le rilasciano → niente attesa circolare, niente deadlock.
- Mai più di 2 thread al lavoro insieme (4 risorse, 2 a testa).

Riepilogo:
- Idea di fondo: per N risorse usabili in mutua esclusione basta inizializzare il semaforo a N invece che a 1 (**sem_init(semaphore, 0, NUM_RESOURCES)**).
- Protocollo: **sem_wait** per acquisire (se le N risorse sono occupate il thread resta in coda), uso della risorsa, **sem_post** per rilasciare (il valore risale oppure parte un thread in coda).
- Risultato: mai più di **NUM_RESOURCES** thread contemporaneamente tra "acquired" e "released".
- Variante 1 (a coppie): **sem_wait** quando **i%2 == 0**, **sem_post** quando **i%2 == 1** oppure all'ultimo work item (**i == num_tasks-1**), altrimenti con **NUM_TASKS** dispari il thread terminerebbe tenendo la risorsa. I semafori POSIX non garantiscono FIFO, quindi può esserci starvation.
- Variante 2 (chiusura del main): se il main distrugge il semaforo mentre i thread son vivi, si ha comportamento indefinito (qui un crash). Due soluzione:
  - un secondo semaforo inizializzato a 0 come contatore dei thread terminati: ogni thread fa **sem_post** alla fine, il main fa **sem_wait** per ogni thread creato, senza busy waiting;

  - **on_exit() + pthread_exit()** nel main: il semaforo viene distrutto alla terminazione del processo, dopo l'ultimo thread.
- Variante alternativa (2 risorse per thread): prendere le risorse una alla volta porta a deadlock (hold and wait + attesa circolare). Si risolve con un mutex (**acquire_mutex**, semaforo a 1) che rende atomica l'acquisizione delle 2 risorse. Il mutex si rilascia appena ottenute entrambe, e il rilascio delle risorse non lo usa.
- Compilazione: **gcc -o scheduler_solution scheduler_solution.c -lpthread**.

INDICE

Laboratorio 01 **1**

Esercizio 1: Processi vs Thread 1

Esercizio 2: Accesso concorrente a variabili condivise 3

Laboratorio 02 **6**

Esercizio 1: Accesso alla sezione critica in mutua esclusione 6

Esercizio 2: Accesso in mutua esclusione a N risorse 9